



Assignment #01

Name: Zara Ahmed

Registration no: Fa25-BCS-060

Course name: Data Structure

Submission date: 21 September, 2026

Instructor: Bilal Bukhari

Code#01

```
#include <iostream>
```

```
#include <cmath>
```

```
using namespace std;
```

```
const int CAP = 20;
```

```
// ArrayList mein ek array aur uska current size hai
```

```
struct ArrayList
```

```
{
```

```
    int data[CAP];
```

```
    int size = 0;
```

```
};
```

```
bool insertEnd(ArrayList &list, int val)
```

```
{
```

```
    if (list.size >= CAP)
```

```
{  
    return false;  
}  
  
list.data[list.size] = val;  
list.size++;  
  
return true;  
}  
  
bool insertBegin(ArrayList &list, int val)  
{  
    if (list.size >= CAP)  
    {  
        return false;  
    }  
  
    for (int i = list.size; i > 0; i--)  
    {  
        list.data[i] = list.data[i - 1];  
    }
```

```
list.data[0] = val;
```

```
list.size++;
```

```
return true;
```

```
}
```

```
bool deletePos(ArrayList &list, int pos)
```

```
{
```

```
    if (pos < 0 || pos >= list.size)
```

```
    {
```

```
        return false;
```

```
    }
```

```
    for (int i = pos; i < list.size - 1; i++)
```

```
    {
```

```
        list.data[i] = list.data[i + 1];
```

```
    }
```

```
list.size--;
```

```
return true;
```

```
}
```

```
void display(const ArrayList &list)
```

```
{
```

```
    for (int i = 0; i < list.size; i++)
```

```
    {
```

```
        cout << list.data[i] << " ";
```

```
    }
```

```
    cout << endl;
```

```
}
```

```
int main()
```

```
{
```

```
    ArrayList list;
```

```
    int *ptr = list.data;
```

```
    int *minPtr = list.data;
```

```
int *maxPtr = list.data;
```

```
// Median value ke element ka address
```

```
int *medPtr = list.data;
```

```
// Special Average ke closest value ka address
```

```
int *closePtr = list.data;
```

```
int sum = 0;
```

```
int closePos;
```

```
double genAvg;
```

```
double spAvg;
```

```
double avgDiff;
```

```
double score;
```

```
insertEnd(list, 18);
```

```
insertEnd(list, 7);
```

```
insertEnd(list, 45);
```

```
insertEnd(list, 11);
```

```
insertEnd(list, 36);
```

```
// Apni registration number ke last two digits yahan likho
```

```
insertEnd(list, 60);
```

```
insertEnd(list, 21);
```

```
insertEnd(list, 13);
```

```
insertEnd(list, 29);
```

```
cout << "Initial ArrayList: ";
```

```
display(list);
```

```
cout << endl;
```

```
ptr = list.data;
```

```
minPtr = list.data;
```

```
maxPtr = list.data;
```

```
sum = 0;
```

```
// Array ko pointer ki help se traverse karna
```

```
for (int i = 0; i < list.size; i++)
```

```
{
```

```
    // *ptr ka matlab:
```

```
    // jis value ko ptr point kar raha hai
```

```
    sum = sum + *ptr;
```

```
    if (*ptr < *minPtr)
```

```
    {
```

```
        // minPtr ko current element ka address de do
```

```
        minPtr = ptr;
```

```
    }
```

```
    if (*ptr > *maxPtr)
```

```
    {
```

```
        maxPtr = ptr;
```



```
}
```

```
ptr++;
```

```
}
```

```
// Pointer ko dereference karke value print karna
```

```
cout << "Minimum Value: " << *minPtr << endl;
```

```
cout << "Maximum Value: " << *maxPtr << endl;
```

```
cout << "Sum: " << sum << endl;
```

```
cout << endl;
```

```
int temp[CAP];
```

```
for (int i = 0; i < list.size; i++)
```

```
{
```

```
    temp[i] = list.data[i];
```

```
}
```

```
for (int i = 0; i < list.size - 1; i++)  
{  
    for (int j = 0; j < list.size - i - 1; j++)  
    {  
        if (temp[j] > temp[j + 1])  
        {  
            int x = temp[j];  
  
            temp[j] = temp[j + 1];  
  
            temp[j + 1] = x;  
        }  
    }  
}
```

```
int mid = list.size / 2;
```

```
// Sorted array se middle value lena
```

```
int medVal = temp[mid];
```

```
ptr = list.data;
```

```
for (int i = 0; i < list.size; i++)  
{  
    // *ptr current value hai  
    if (*ptr == medVal)  
    {  
        // Median ke address ko medPtr mein store karo  
        medPtr = ptr;  
  
        break;  
    }  
  
    // Next element  
    ptr++;  
}
```

```
cout << "Median Value: " << *medPtr << endl;
```

```
cout << endl;
```

```
genAvg = (double)sum / list.size;
```

spAvg =

((double)*minPtr + *medPtr + *maxPtr) / 3;

// Pointer ko first element par set karo

ptr = list.data;

closePtr = list.data;

double closeDist =

fabs(*closePtr - spAvg);

// Puri array traverse karo

for (int i = 0; i < list.size; i++)

{

// Current value aur Special Average ka difference

double curDist =

fabs(*ptr - spAvg);

```

if (curDist < closeDist)
{
    // New smallest distance save karo
    closeDist = curDist;

    // Current element ka address save karo
    closePtr = ptr;
}

// Next element
ptr++;
}

closePos = closePtr - list.data;

cout << "General Average: " << genAvg << endl;

cout << "Special Average: " << spAvg << endl;

cout << "Closest Value: " << *closePtr << endl;

cout << "Position of Closest Value: "

```

```
<< closePos << endl;
```

```
cout << endl;
```

```
avgDiff =
```

```
    fabs(genAvg - spAvg);
```

```
score =
```

```
    fabs(*closePtr - genAvg)
```

```
    +
```

```
    fabs(*closePtr - spAvg)
```

```
    +
```

```
    avgDiff;
```

```
cout << "Difference Between Averages: "
```

```
    << avgDiff << endl;
```

```
cout << "Final Score: "
```

```
    << score << endl;
```

```
cout << endl;
```

```
closePos = closePtr - list.data;
```

```
deletePos(list, closePos);
```

```
cout << "ArrayList After Deletion: ";
```

```
display(list);
```

```
cout << endl;
```

```
int roundAvg =
```

```
    (int)round(spAvg);
```

```
insertBegin(list, roundAvg);
```

```
cout << "Final ArrayList After Insertion: ";
```

```
display(list);
```

```
return 0;
```

```
}
```

Output:

```
C:\Users\SHOAIB\Documents' x + v - □  
Initial ArrayList: 18 7 45 11 36 8 21 13 29  
Minimum Value: 7  
Maximum Value: 45  
Sum: 188  
Median Value: 18  
General Average: 20.8889  
Special Average: 23.3333  
Closest Value: 21  
Position of Closest Value: 6  
Difference Between Averages: 2.44444  
Final Score: 4.88889  
ArrayList After Deletion: 18 7 45 11 36 8 13 29  
Final ArrayList After Insertion: 23 18 7 45 11 36 8 13 29  
  
-----  
Process exited after 0.595 seconds with return value 0  
Press any key to continue . . . |
```